



Venue	GURKEY NEST									
Seat Number	4	1	9							
Student Number	1	3	3	7	8	3	3	7		
Family Name	MACLOC									
First Name	BARRY (BAZZA, LEGEND)									

This paper is for St Lucia Campus students.

For Examiner Use Only

Question	Mark
----------	------

- This is a Closed Book examination - specified written materials permitted
- Casio FX82 series or UQ approved and labelled calculator only
- During Planning Time - Students are encouraged to review and plan responses to the exam questions
- This examination paper will be released to the Library

(No electronic aids are permitted e.g. laptops, phones)

One A4 sheet of handwritten or typed notes double sided is permitted

Additional exam materials (e.g. answer booklets, rough paper) will be provided upon request.

If you believe there is missing or incorrect information impacting your ability to answer any question, please state this when writing your answer.

The exam is worth 100 marks (50% of your final grade).

[illegible]

Total _____

Question 0. [0 MARK]

Write your name and student number on the cover sheet of the exam paper. Then, write your student number on the footer of each page in the *Student ID* area.

Question 1. [20 MARKS]

For each of the following 10 **statements**, circle either **T** (for True) or **F** (for False) and **briefly** justify your answer in the space provided. Each problem is worth 2 points: 1 point for the correct answer, and 1 point for the justification. The justification should be either a short sentence or a drawing.

Part (a) [2 MARKS]

Statement: $n! \in O(2^n)$

~~$n!$~~ $n! > 2^n$ for all $n > 4$,
so $n! \notin O(2^n)$

☐ T ☒ F

Part (b) [2 MARKS]

Statement: Mergesort, which is $O(n \log n)$, will always run faster than Bubblesort, which is $O(n^2)$.

Very small lists - Bubblesort may run faster
Sorted lists - Bubblesort may run faster

☐ T ☒ F

Part (c) [2 MARKS]

Let $f(n) = 1000n^3 \times 57n^2 + 4^n + 1337$.

Statement: $f(n) \in \Omega(2^n)$.

$f(n) = O(4^n)$ using simplification rules.
 $2^n < 4^n$ for all $n > 0$, so $f(n) \in \Omega(2^n)$

☒ T ☐ F

Part (d) [2 MARKS]

A hash table, implemented with separate chaining, is being used to store $n = 100$ elements with $N = 100$ chains.

Statement: Inserting key k will result in a collision.

All $n = 100$ elements could be held in a single chain, so $h(k)$ may not collide with that (single) chain. NOTE

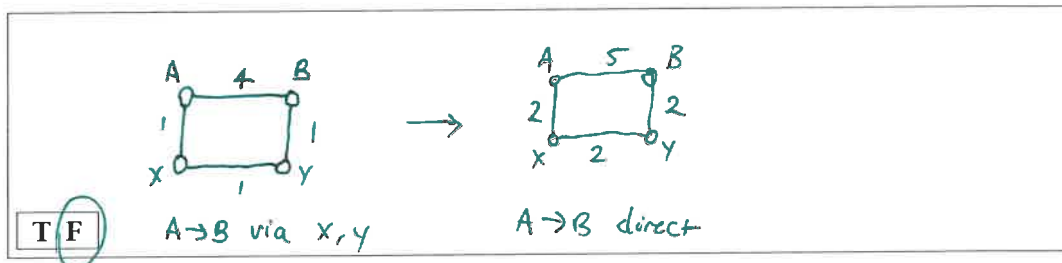
Another possible answer could be: "True, as 100 chains implies each chain containing ≥ 1 element, so a collision cannot be avoided"

T F

Part (e) [2 MARKS]

Let $G = (V, E)$ be an undirected graph with positively weighted edges. Dijkstra's algorithm has been used to find the shortest path between $u \in V$ and all other vertices in G .

Statement: Adding a weight of one to each edge $e \in E$ will not change any of the shortest paths computed from u on the original graph G .

**Part (f)** [2 MARKS]

Given a suffix array SA built over text T with $n = |T|$.

Statement: The total number of unique occurrences of pattern P in T can be found in $\mathcal{O}(\log n)$ time.

SA indexes suffixes of T . Can find the total number of unique occurrences of P in T via binary searching SA twice. BUT each search costs $\mathcal{O}(|P| \log n)$ since we must check the pattern at each step.

T F

Question 2. [4 MARKS]

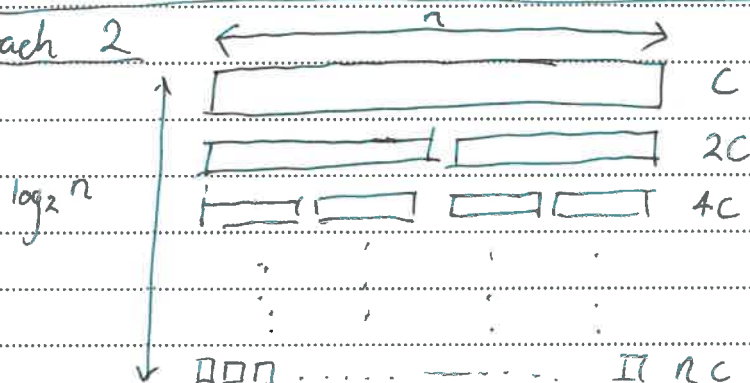
Solve the following recurrence relation. Make sure you show/draw your work. You should clearly state the complexity of T .

$$T(n) = \begin{cases} \mathcal{O}(1), & n = 1, \\ 2T(n/2) + \mathcal{O}(1), & n > 1. \end{cases}$$

Approach 1

$$\begin{aligned} & 2T(n/2) + C \\ &= 2[2T(n/4) + C] + C = 4T(n/4) + 3C \\ &= 2[2[2T(n/8) + C] + C] + C = 8T(n/8) + 7C \\ &\vdots \\ &= 2^k T(n/2^k) + (2^k - 1)C \\ &\text{WANT } T(n/2^k) \text{ to be } T(1) \text{ so let } k = \log_2 n \\ &\Rightarrow 2^{\log_2 n} T(n/2^{\log_2 n}) + (2^{\log_2 n} - 1)C \\ &= n T(n/n) + (n-1)C \\ &= n \cdot C + (n-1)C \\ &= \mathcal{O}(n) \end{aligned}$$

Approach 2



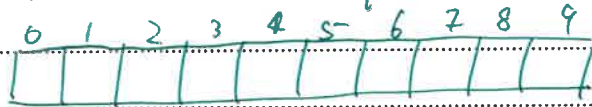
- Must be $\log_2 n$ total levels, as we split $n/2$ each step.
- 2 calls per split, constant work each
- $\therefore$ Summing work over all calls is $\mathcal{O}(n)$

Question 3. [15 MARKS]**Part (a)** [5 MARKS]

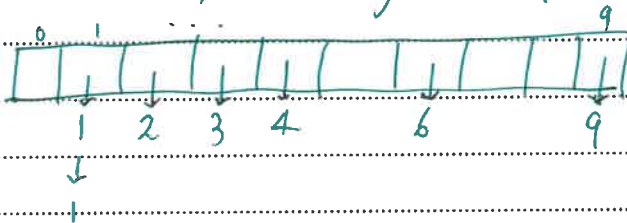
Explain how **bucket sort** works. You may use either pseudocode, or describe the algorithm in plain language. You may draw pictures to assist with your explanation.

• Bucket sort is a sorting algorithm that puts each element in the input array into its corresponding "bucket", where buckets are used to capture specific ranges of values.

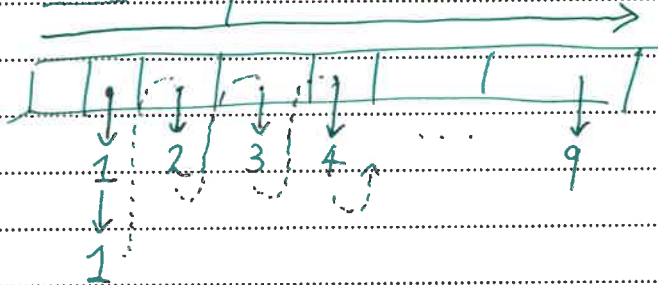
For example, let input $A = 2, 4, 6, 1, 3, 9, 1$.
We can define $N = 10$ unique buckets, 0 to 9.



Each element in A is placed into its corresponding bucket, in order of the original sequence.



Next, buckets iterated low-to-high to pick up the sorted sequence.



$\Rightarrow 1, 1, 2, 3, 4, 6, 9$.

Note the sort keeps elements within each bucket "stable" in terms of the original input sequence A .

Part (b) [5 MARKS]

What is the worst-case space complexity of **bucket sort**, and when does it arise? Provide an example.

- Input array/sequence has n elements.
- Define N unique buckets.

Total space is then $O(n + N)$

An example: $A = 1, 2, 20, 100$

$N = 100$ buckets

Space = 4 elements (A) + 100 buckets,
 $O(n + N)$

Question 4. [10 MARKS]**Part (a)** [8 MARKS]

Given a singly linked list (as defined below) and a value k , partition the linked list such that all nodes less than x appear *before* k , and all nodes greater than k appear *after* k .

As a secondary constraint, you must preserve the relative order of elements within each side of the partition.

Example Execution:

```
input = [2, 7, 3, 6, 4, 2, 2]
linked_list = SinglyLinkedList(input) # convert python list to linked list
k = 4
linked_list.partition(k)
print(linked_list)
2 -> 3 -> 2 -> 2 -> 4 -> 7 -> 6
```

Another Example Execution:

```
input = [1, 2, 3, 4, 3, 2, 1]
linked_list = SinglyLinkedList(input) # convert python list to linked list
k = 4
linked_list.partition(k)
print(linked_list)
1 -> 2 -> 3 -> 3 -> 2 -> 1 -> 4
```

Your solution goes on the next page...

```
1 class SingleNode:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6     # You may assume typical getters/setters exist
7
8 class SinglyLinkedList:
9     def __init__(self):
10         self.head = None
11         self.size = 0
12
13     # You may assume typical getters/setters exist
```

Your Solution:

```

1      # continued inside the SinglyLinkedList class definition
2      def partition(self, k):
3          head = self.get_head() # assume at least 2 elements
4          l_head = None, l_cur = None
5          e_head = None, e_cur = None
6          g_head = None, g_cur = None
7          cur = head
8          while cur is not None:
9              if cur.get_data() < k:
10                 if l_cur is not None:
11                     l_cur.next = cur
12                 else:
13                     l_head = cur
14                     l_cur = cur
15                     l_cur = cur
16                 cur = cur.next
17                 l_cur.next = None
18             elif cur.get_data() == k:
19                 # Repeat helper-A logic on e-head, e-cur
20             else:
21                 # Repeat helper-A logic on g-head, g-cur
22
23         a
24         # Now we have 3 linked lists: one with lower (l-head), equal (e-head),
25         # and one greater (g-head). Lets connect them
26         if l_head is not None:
27             self.head = l_head
28         elif e_head is not None:
29             self.head = e_head
30         else:
31             self.head = g_head
32
33         # Do it here, but out of space :

```

Call this "helper A"

Get the linked list head sorted out. l-cur
Need similar logic to connect ~~l-head~~ to e-head, ~~e-head~~ e-cur to g-head.

NB: This was done under "real" time pressure, ~5-10 mins.
Could be incorrect in some cases, no time to check i corner

This is more or less what I would expect from students.

Part (b) [2 MARKS]

What is the worst-case time and space complexity of your partition function, and when does it occur?

Algorithm does a linear pass through the linked list, and uses only constant additional space for some "pointer" variables. Time is $O(n)$ worst-case, space is $O(1)$ additional, worst-case. The algorithm will always take $O(n) \Rightarrow \Theta(n)$ time, $O(1)$ space.

Question 5. [8 MARKS]**Part (a)** [5 MARKS]

Given the following text T , build a minimum redundancy code based on the frequency of each character. Make sure you show your work. You may ignore the space character.

$T = y e e y u s y e e t e d e t$

c	d	e	s	+	u	y
f(c)	1	6	1	3	1	2

w =	1	1	1	2	3	6	Length	Count
	2	3	5	8	14		1	1
							2	1
							3	1
							4	1
							5	2

d: 00000
 s: 00001
 u: 0001
 y: 001
 +: 01
 e: 1

Total cost of code
 $6 \cdot 1 + 3 \cdot 2 + 2 \cdot 3$
 $+ 4 + 5 + 5$
 $= 32 \text{ bits}$

Not required

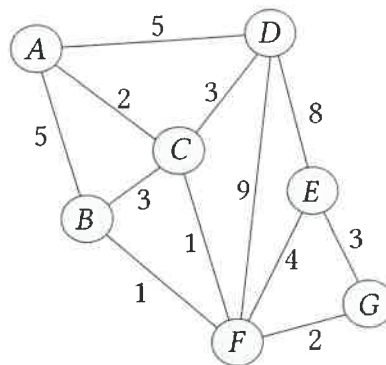
Part (b) [3 MARKS]

Using your code from above, encode the string $S = yet$:

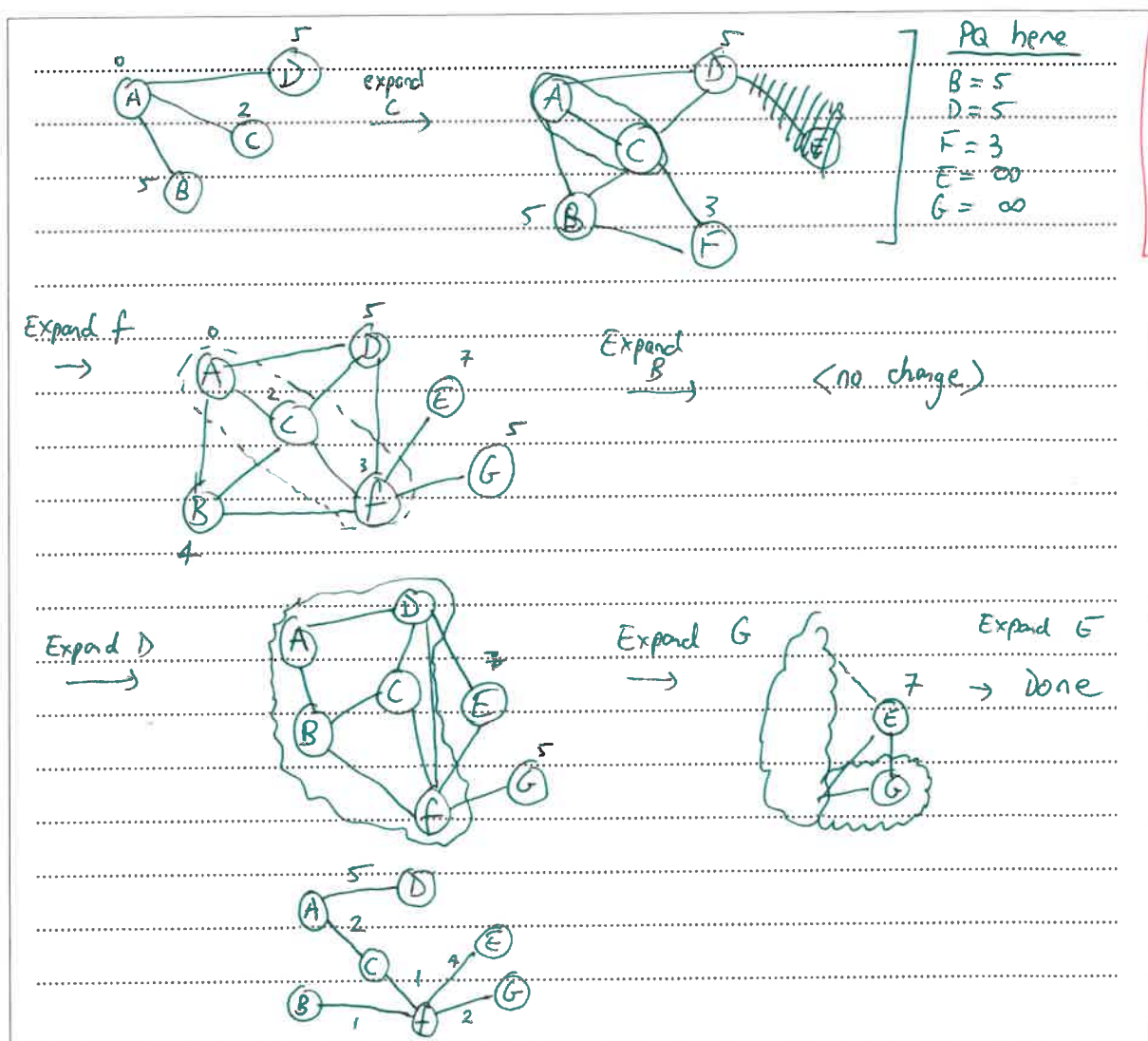
$\frac{001}{y} \quad \frac{1}{e} \quad \frac{01}{t} \Rightarrow 001101$

Question 6. [10 MARKS]

The following questions are based on the graph below.

**Part (a)** [4 MARKS]

Run Dijkstra's algorithm from vertex A to find the shortest path between A and all other nodes. Show your work.



Worth showing the PQ at least once or twice...

Part (b) [4 MARKS]

Run Prim-Jarnik on the graph to find a minimum spanning tree.

① Pick node C (any will do)

② PQ is:

A:	2
B:	3
C:	0
D:	3
E:	INF
F:	1 ← Smallest
G:	INF

③ Remove F from queue, add edge $C \leftrightarrow F$ into MST.

④ Update PQ values from F: E becomes 4, G becomes 2, B 1

⑤ Remove B from queue, add edge $F \leftrightarrow B$ into MST.

⑥ Update PQ values from B: Nothing to do ...

Continue until PQ empty.

MST:

~~NOTE~~
NOTE THE ALTERNATIVE TO PART (A). ANY WORKING IS OK BUT WE MUST BE ABLE TO CLEARLY UNDERSTAND IT.

Part (c) [2 MARKS]

Generally speaking, does the shortest path from some vertex v to all other nodes — as computed via Dijkstra's — yield the same result as the MST computed by Prim-Jarnik? Why or why not?

No. See counter-example:

• shortest path $C \rightarrow B$ costs 2, $C \rightarrow A$ costs 2

• MST(G) is OR

$\therefore$ Dijkstra and Prim compute different things...

Question 7. [7 MARKS]

¹ **Recommendation:** Only attempt once you have tried the other questions.

Let S be a set of n entries of the form $\langle k, v \rangle$ (where k is the key, and v is the corresponding value). Your task is to design and **describe** a data structure that can process S that can support the following operations:

- **Build(S):** Given set S , build the data structure.
- **Query($\langle Q_k, Q_v \rangle$):** Given a query key, Q_k , and a query value, Q_v , return the number of $\langle k, v \rangle$ pairs in S such that $k = Q_k$ and $v > Q_v$. In other words, given a query key/value pair, return the number of items in S that have the same key as the query key, and have a larger value than the query value.

Constraints For full marks, you must support Build in $\mathcal{O}(n \log f)$ time, where f is the maximum number of times a given key appears in S . You must support Query in $\mathcal{O}(\log f)$ expected time.

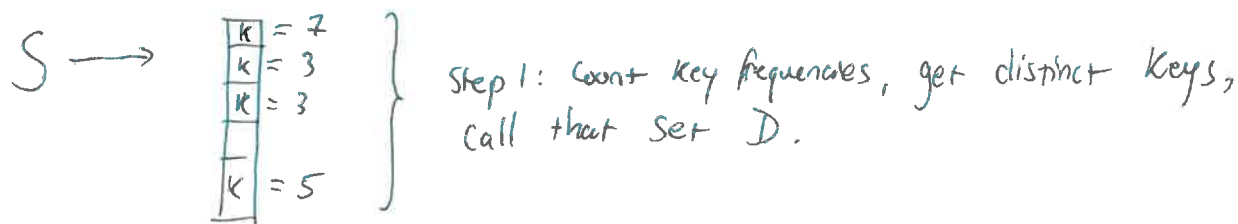
First, iterate each key k in S and hash it into a hash table to compute the frequency of each key.
 $\triangleright$ Hash-table lookup is $\mathcal{O}(1)$ in expectation, so cost is $\mathcal{O}(n)$ expected. Let the set of distinct keys be D .
 Next, create another hash table on D in $\mathcal{O}(n)$ time, and for each unique key k in D , initialize an array A_k with length equal to frequency (A_k) [computed in step 1].
 In A_k , store all values v where (k, v) is a pair in S .
 Sort each A_k in ascending order; $\mathcal{O}(|A_k| \log |A_k|)$ time.
 $\triangleright$ Total construction time: $\sum_{k=0}^K |A_k| \log f \in \mathcal{O}(n \log f)$
 $\triangleright$ Space is $\mathcal{O}(n)$

To answer query $\langle Q_k, Q_v \rangle$, look up the hash table for key Q_k : if not found, return 0. Otherwise, binary search A_k for Q_v in $\mathcal{O}(\log f)$ time, and return the index $|A_k| - \text{index of binary search}$.

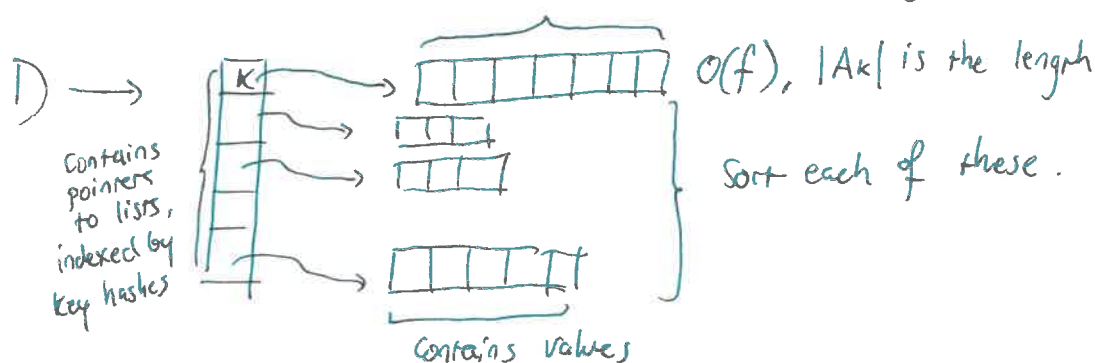
Query time is $\mathcal{O}(1)$ expected, plus $\mathcal{O}(\log f)$ worst-case yielding the desired $\mathcal{O}(\log f)$ expected.

¹Inspired by a question posted by Prof. Yufei Tao, who taught COMP3506/7505 at UQ in the mid 2010's

Solution to 7 (visually)

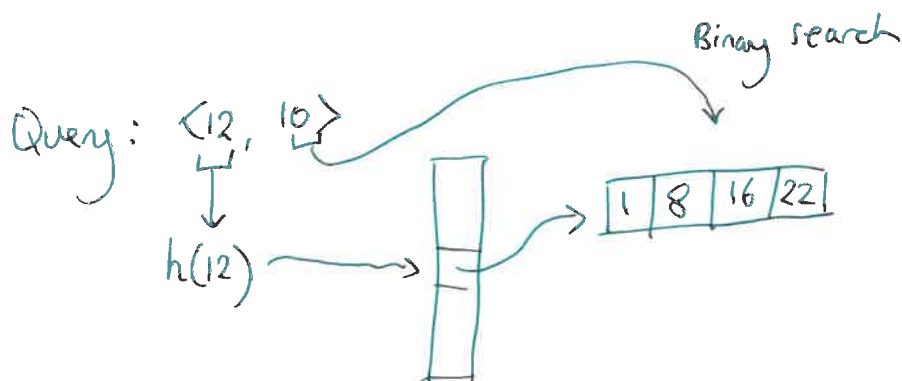
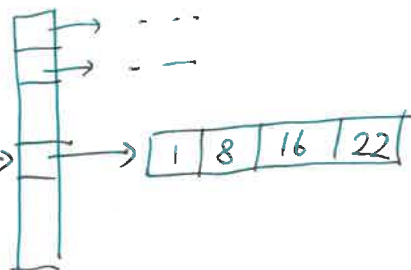


sort this, in $O(|A_k| \log |A_k|)$



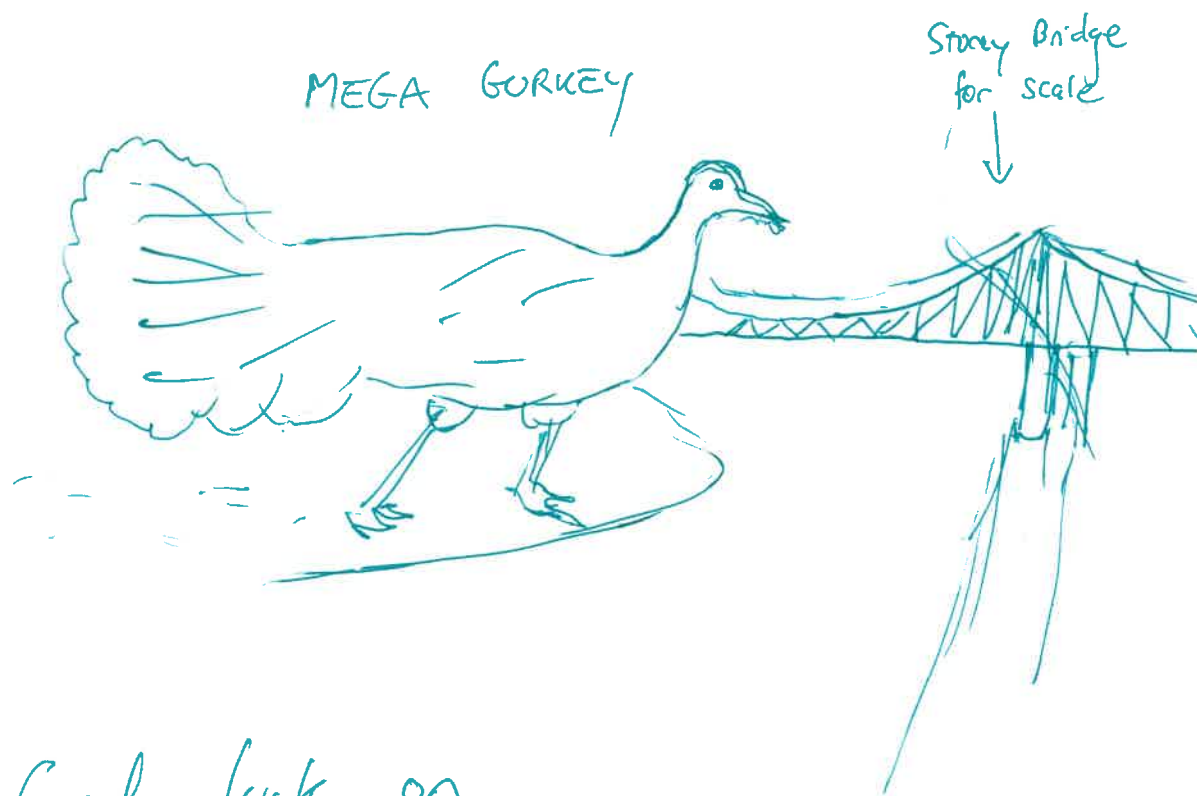
Built structure
(example)

key = 12
 $h(12)$



What makes this question tricky, apart from the creative ideas, is that we are talking about expected time, not worst-case.

END OF EXAMINATION



Good luck on
Friday, everyone :)

